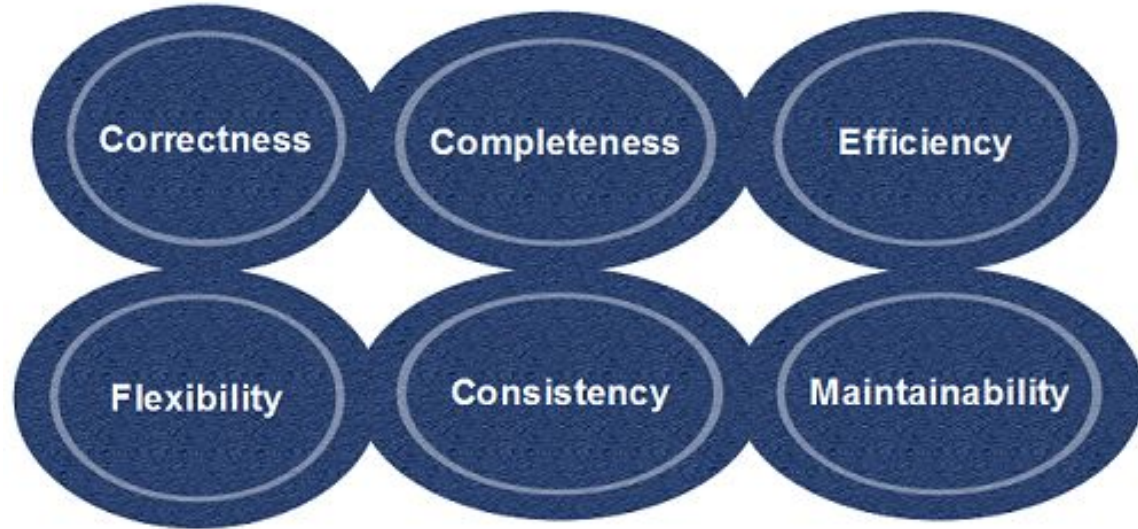


Software Design

Introduction

- + Software design is an iterative process through which requirements are translated into a blueprint for constructing the software.
- + It consist of set of principles, concepts and practices that lead to the development of high quality system

Objectives of Software Design



Objectives of Software Design

Objectives of Software Design

1. **Correctness:** Software design should be correct as per requirement.
2. **Completeness:** The design should have all components like data structures, modules, and external interfaces, etc.
3. **Efficiency:** Resources should be used efficiently by the program.
4. **Flexibility:** Able to modify on changing needs.
5. **Consistency:** There should not be any inconsistency in the design.
6. **Maintainability:** The design should be so simple so that it can be easily maintainable by other designers.

Software design principles

- + Software design principles are a set of guidelines that helps developers to make a good system design.
- + Software design principles are concerned with providing means to handle the complexity of the design process effectively.

Software design principles

Some of the commonly followed design principles are as following:

1. Software design should correspond to the analysis model
2. Choose the right programming paradigm
3. Software design should be uniform and integrated
4. Software design should be flexible

Software design principles

5. Software design should ensure minimal conceptual (semantic) errors
6. Software design should be structured to degrade gently
7. Software design should represent correspondence between the software and real-world problem
8. Software reuse
9. Designing for testability
10. Prototyping

Software Design Concepts

1. **Abstraction:** hide Irrelevant data
2. **Architecture:** design a structure of something
3. **Pattern:** a repeated form
4. **Modularity:** subdivide the system
5. **Information hiding:** hide the information
6. **Refinement:** removes impurities
7. **Refactoring:** reconstruct something
8. **Structural partitioning:** Structure of the program can be partitioned
9. **Concurrency:** control the concurrent execution of process

Design Concepts – 1. Abstraction

- + Abstraction refers to a powerful design tool, which allows software designers to consider components at an abstract level, while neglecting the implementation details of the components.
- + There are three commonly used abstraction mechanisms in software design, namely, functional abstraction, data abstraction and control abstraction.
- + All these mechanisms allow us to control the complexity of the design process

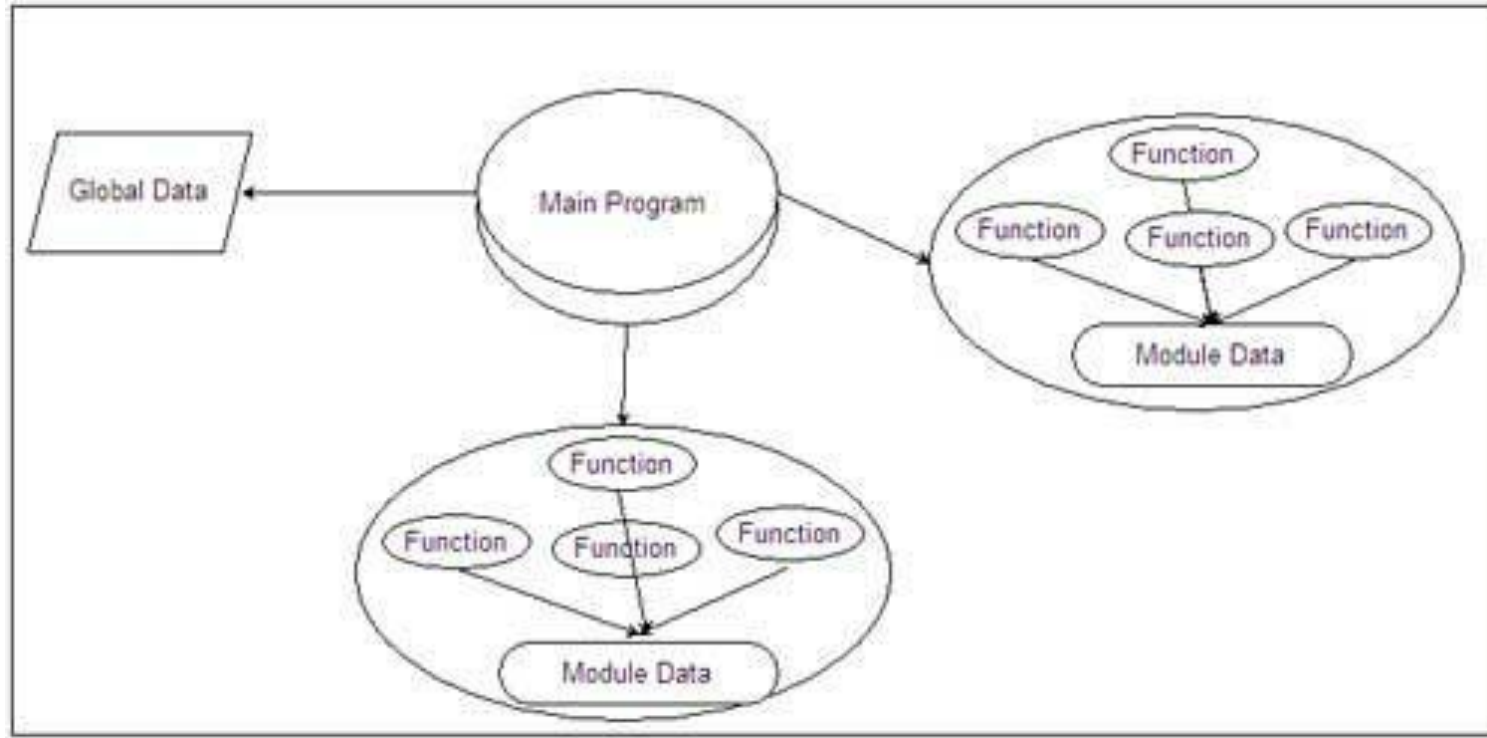
Design Concepts – 1. Abstraction

1. **Functional abstraction:** It can be generalized as collections of subprograms referred to as 'groups.
2. **Data abstraction:** This involves specifying data that describes a data object.
3. **Control abstraction:** This states the desired effect, without stating the exact mechanism of control.

Design Concepts – Modularity

- + Modularity is achieved by dividing the software into uniquely named and addressable components, which are also known as modules.
- + Modularizing a design helps to plan the development in a more effective manner, accommodate changes easily, conduct testing and debugging effectively and efficiently, and conduct maintenance work without adversely affecting the functioning of the software.

Design Concepts – Modularity



SOLID PRINCIPLES

- + The SOLID principles are five essential guidelines that enhance software design, making code more maintainable and scalable.
- + The SOLID principles help in enhancing loose coupling. Loose coupling means a group of classes are less dependent on one another.
- + Loose coupling helps in making code more reusable, maintainable, flexible and stable. Now let's discuss one by one these principles...
- + Loosely coupled classes minimize changes in your code when some changes are required in some other code.

SOLID Design Principles

- + **SOLID** is an acronym for five object-oriented design principles that help you build software that is **maintainable, scalable, testable, and flexible**. They were introduced by Robert C. Martin (also known as “Uncle Bob”).
- + S — Single Responsibility Principle (SRP) A class should do **one thing only** and do it well.
- + O — Open/Closed Principle (OCP) **Software entities should be open for extension, but closed for modification**. You should be able to add new functionality **without modifying existing code**.
- + L — Liskov Substitution Principle (LSP) : **Subtypes must be substitutable for their base types** . If class B is a subclass of A, you should be able to use B anywhere A is expected.

+ I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

Better to have many small, specific interfaces than one large general-purpose interface

+ D — Dependency Inversion Principle (DIP)

Depend on abstractions, not on concrete implementations.

High-level modules should not depend on low-level modules.

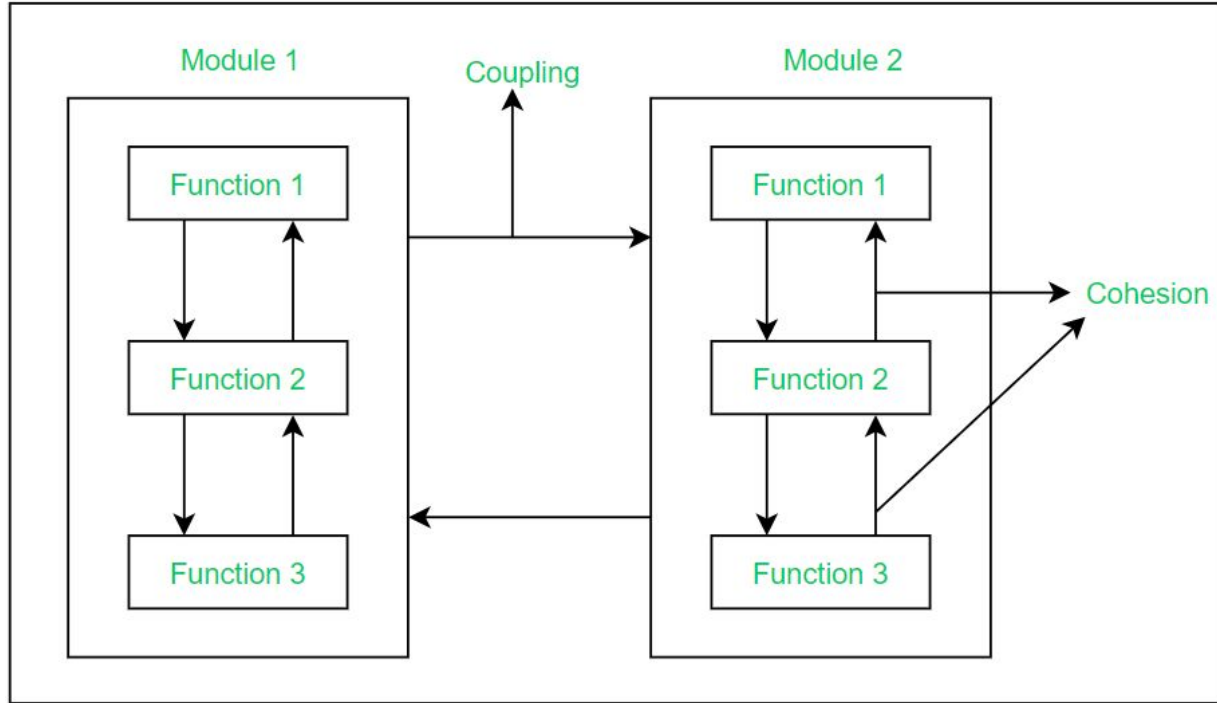
Effective modular design

- + Effective modular design can be achieved if the partitioned modules are separately solvable, modifiable as well as easy to compile.
- + Here separate compilation of modules means that after making changes in a module there is no need of recompiling the whole software system.

Effective modular design

- + In order to build a software with effective modular design there is a factor “**Functional Independence**” which comes into play.
- + The Functional Independence means a function is atomic in nature so that it performs only a single task of the software without or with least interaction with other modules.
- + Independence of modules of a software system can be measured using 2 criteria : **Cohesion, and Coupling**.

Effective modular design



A good software design requires **high cohesion** and **low coupling**.

Cohesion

- + Cohesion is a measure of the degree to which the elements of the module are functionally related.
- + It is the degree to which all elements directed towards performing a single task are contained in the component.
- + A good software design will have high cohesion.



Types of Cohesion

1. **Functional Cohesion:** Functional cohesion is said to exist if different elements of a module cooperate to achieve a single function.
2. **Sequential Cohesion:** An element outputs some data that becomes the input for other element, i.e., data flow between the parts.
3. **Communicational Cohesion:** Two elements operate on the same input data or contribute towards the same output data.

Types of Cohesion

4. **Procedural Cohesion:** components within module ensure the order of execution. Actions are still weakly connected and unlikely to be reusable.
5. **Temporal Cohesion:** This cohesion contains the code for initializing all the parts of the system or lots of different activities occur, all at time of initialization.

Types of Cohesion

6. **Logical Cohesion:** The elements are logically related and not functionally. All the code for these functions is in the same component. Operations are related, but the functions are significantly different.
7. **Coincidental Cohesion:** The elements are not related i.e., it performs a set of function or tasks that relate to each other very loosely. The elements have no conceptual relationship other than location in source code. It is accidental and the worst form of cohesion.

Coupling

- + **Coupling** between two modules is a measure of the degree of interaction or interdependence between the two modules.
- + A module having low coupling and high cohesion is said to be **functionally independent** of other modules.



Types of Coupling

1. **Data Coupling:** If the dependency between the modules is based on the fact that they communicate by passing only data, then the modules are said to be data coupled. In data coupling, the components are independent to each other and communicating through data.
2. **Stamp Coupling:** In stamp coupling, the complete data structure is passed from one module to another module.

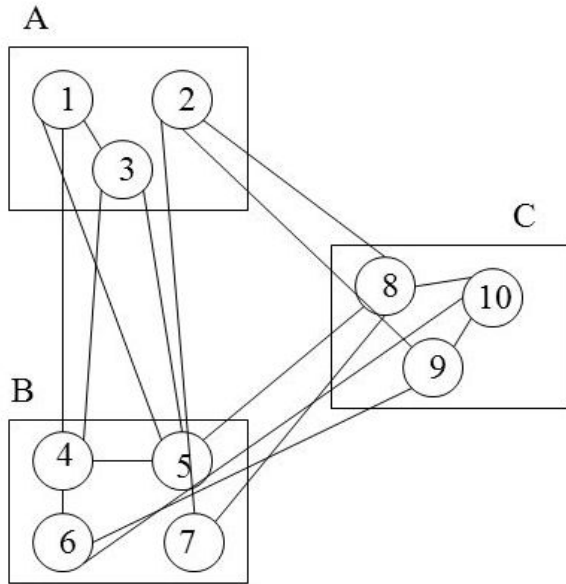
Types of Coupling

3. **Control Coupling:** Control coupling exists between two modules if data from one module is used to direct the order of instructions executed in another. An example of control coupling is a flag set in one module and tested in another module.
4. **External Coupling:** In external coupling, the modules depend on other modules, external to the software being developed or to a particular type of hardware.

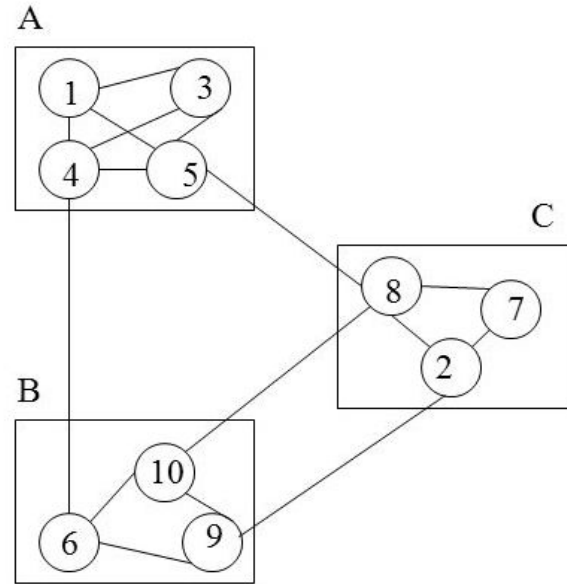
Types of Coupling

5. **Common Coupling:** The modules have shared data such as global data structures. The changes in global data mean tracing back to all modules which access that data to evaluate the effect of the change.
6. **Content Coupling:** In a content coupling, one module can modify the data of another module or control flow is passed from one module to the other module. This is the worst form of coupling and should be avoided.

Cohesion vs Coupling



Bad modularization:
low cohesion, high coupling



Good modularization:
high cohesion, low coupling

Cohesion vs Coupling

COUPLING	COHESION
It refers to relationship between two or more modules	It refers to relationship between two or more elements within a module
It focuses on measuring a module's dependency with other modules	It focuses on measuring a module's functional strength
It is ideal to have low coupled modules which states dependency between modules	It is ideal to have a module with high cohesiveness which states implementation of a particular feature with low or no communication with other module
It is based on communication between two modules. Therefore, it is known as inter-module concept	It is based on communication between two elements or fields within a single module. Therefore, it is known as intra-module concept
There are 6 types of coupling: Data Coupling, Stamp Coupling, Control Coupling, External Coupling, Common Coupling and Content Coupling	There are 8 types of cohesion: Functional Cohesion, Sequential Cohesion, Communicational Cohesion, Procedural Cohesion, Temporal Cohesion, Logical Cohesion and Coincidental Cohesion
It is not possible to achieve no coupling among all modules	It is possible to achieve full cohesive modules

How many times have you had design dejaavu—that feeling that you've solved a problem before but not knowing exactly where or how? If you could remember the details of the previous problem and how you solved it, then you could reuse the experience instead of rediscovering it. However, we don't do a good job of recording experience in software design for others to use. The purpose of design patterns is to record experience in designing object-oriented software

Design Patterns

- When experts work on a particular problem, it is unusual for them to tackle it by inventing a new solution that is completely distinct from existing ones. They often recall a similar problem they have already solved, and reuse the essence of its solution to solve the new problem. This kind of 'expert behaviour', the thinking in problem-solution pairs, is common to many different domains. It is a natural way of coping with any kind of problem or social interaction

Design Patterns

- A pattern addresses a recurring design problem that arises in specific design situations, and presents a solution to it.
- Patterns document existing, well-proven design experience.
- Patterns identify & and specify abstractions that are above the level of single classes and instances, or of components
- Patterns provide a common vocabulary and understanding for design principles
- Patterns are a means of documenting software architectures.
- Patterns support the construction of software with defined properties.

Design Patterns

- Patterns help you build complex and heterogeneous software architectures.
- A pattern provides a scheme for a generic solution to a family of problems, rather than a prefabricated module that can be used 'as is'.
- Patterns help solve problems, but they do not provide complete solutions.
- Patterns help you to manage software complexity.

Design Pattern

- A pattern for software architecture describes a particular recurring design problem that arises in specific design contexts, and presents a well-proven generic scheme for its solution. The solution scheme is specified by describing its constituent components, their responsibilities and relationships, and the ways in which they collaborate.

These patterns solve specific design problems and make object-oriented designs more flexible, elegant, and ultimately reusable. They help designers reuse successful designs by basing new designs on prior experience.

Design Patterns

- In software engineering, **design patterns** are reusable solutions to common design problems in object-oriented programming.
- The most popular classification comes from the book **Design Patterns: Elements of Reusable Object-Oriented Software** by the Gang of Four (GoF). They introduced the 23 classic design patterns in 1994.

1. Erich Gamma

- Swiss computer scientist
- Worked on Eclipse IDE
- Major contributor to design patterns concept

2. Richard Helm

- Australian computer scientist
- Worked in object-oriented modelling

3. Ralph Johnson

- Professor at University of Illinois
- Expert in object-oriented programming

4. John Vlissides

- Known as the “Patterns Wizard”
- Worked at IBM Research

- + The pattern is not a specific piece of code, but a general concept for solving a particular problem. You can follow the pattern details and implement a solution that suits the realities of your own program.

Types of Design Patterns

- + In Software Engineering (especially in the context of the famous book by **Gang of Four** — Design Patterns: Elements of Reusable Object-Oriented Software), design patterns are classified into **three main types**:

- + **Creational Design Patterns** : Focus on **object creation mechanisms**

These patterns deal with how objects are created in a flexible and reusable way.

Purpose:

- + Hide object creation logic
- + Make system independent of how objects are created
- + Improve flexibility

Examples: Singleton, Factory Method, Abstract ,Factory, Builder, Prototype

Types of Design Patterns

Structural Design Patterns : Focus on **class and object structure**

These patterns deal with how classes and objects are composed to form larger structures.

- ◆ **Purpose:**

- + Organize classes
- + Reduce complexity
- + Make structure flexible

Examples: Adapter, Bridge, Composite, Decorator, Façade, Proxy, Flyweight

Types of Design Patterns

- + **Behavioral Design Patterns** : Focus on **communication between objects**

These patterns define how objects interact and distribute responsibilities.

- ◆ **Purpose:**

- + Improve communication
 - + Reduce tight coupling
 - + Increase flexibility in behavior

Example: Observer , Strategy , Command , State, Mediator , Template Method , Chain of Responsibility , Iterator , Memento , Interpreter , Visitor

Creational Design Patterns

- + Creational design patterns deal with **object creation mechanisms**. They help make a system **independent of how its objects are created, composed, and represented**.
- + **Singleton**
- + **Factory Method**
- + Abstract Factory
- + Builder
- + Prototype

Singleton Pattern

- + To create object only once & reuse it again and again.
- + Ensures that a class has **only one instance** and provides a **global access point** to it.
- + How do we ensure that a class has only one instance and that the instance is easily accessible?
- + A better solution is to make the class itself responsible for keeping track of its sole instance . The class can ensure that no other instance can be created (by intercepting requests to create new objects), and it can provide a way to access the instance.

Singleton Pattern

```
Public class Singleton {  
    private static Singleton s;    //static method can be called without object of  
                                   the class  
    private Singleton()            //constructor cannot be called as it is  
                                   private cannot create object  
    { System.out.println("Object Created");  
    }  
    public static Singleton getSingleton() {    //to call method without  
                                                object , make it static but    it can be  
invoked many times  
        if(s == null) {  
            s= new Singleton();  
        }    return s;  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Singleton obj1 = Singleton.getSingleton();  
        Singleton obj2 = Singleton.getSingleton();  
  
        System.out.println(obj1 == obj2); // true  
    }  
} //both objects are same
```

FactoryPattern

- + In Factory pattern, we create object without exposing the creation logic to the client and refer to newly created object using a common interface.
- + Define an interface for creating an object , but let subclasses decide which class to instantiate . Also known as Virtual Constructor
- + Child class objects are created based on input

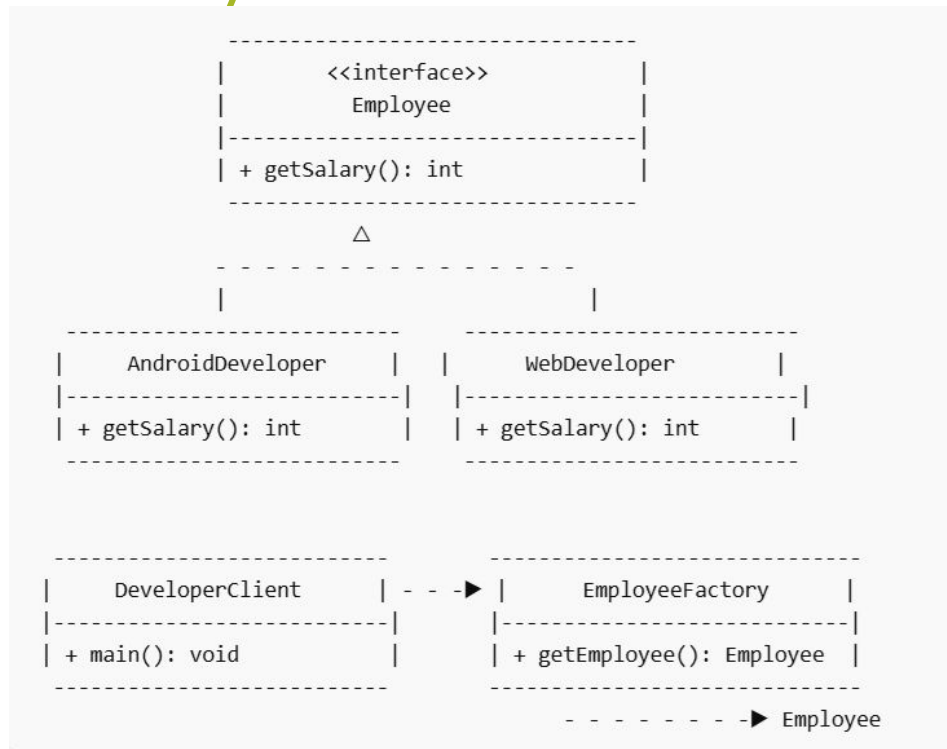
Components:

- **Product (Interface)**
- **Concrete Products (Classes implementing interface)**
- **Factory Class**
- **Client**

e.g. A company (Client) wants to hire developers.

Instead of directly hiring a specific type (Android/Web), it goes through an HR

Factory Method



The **Factory Design Pattern** to reduce tight coupling between the client and concrete classes.

```
Interface Employee{                                //interface  
    int getSalary();}
```

//child class

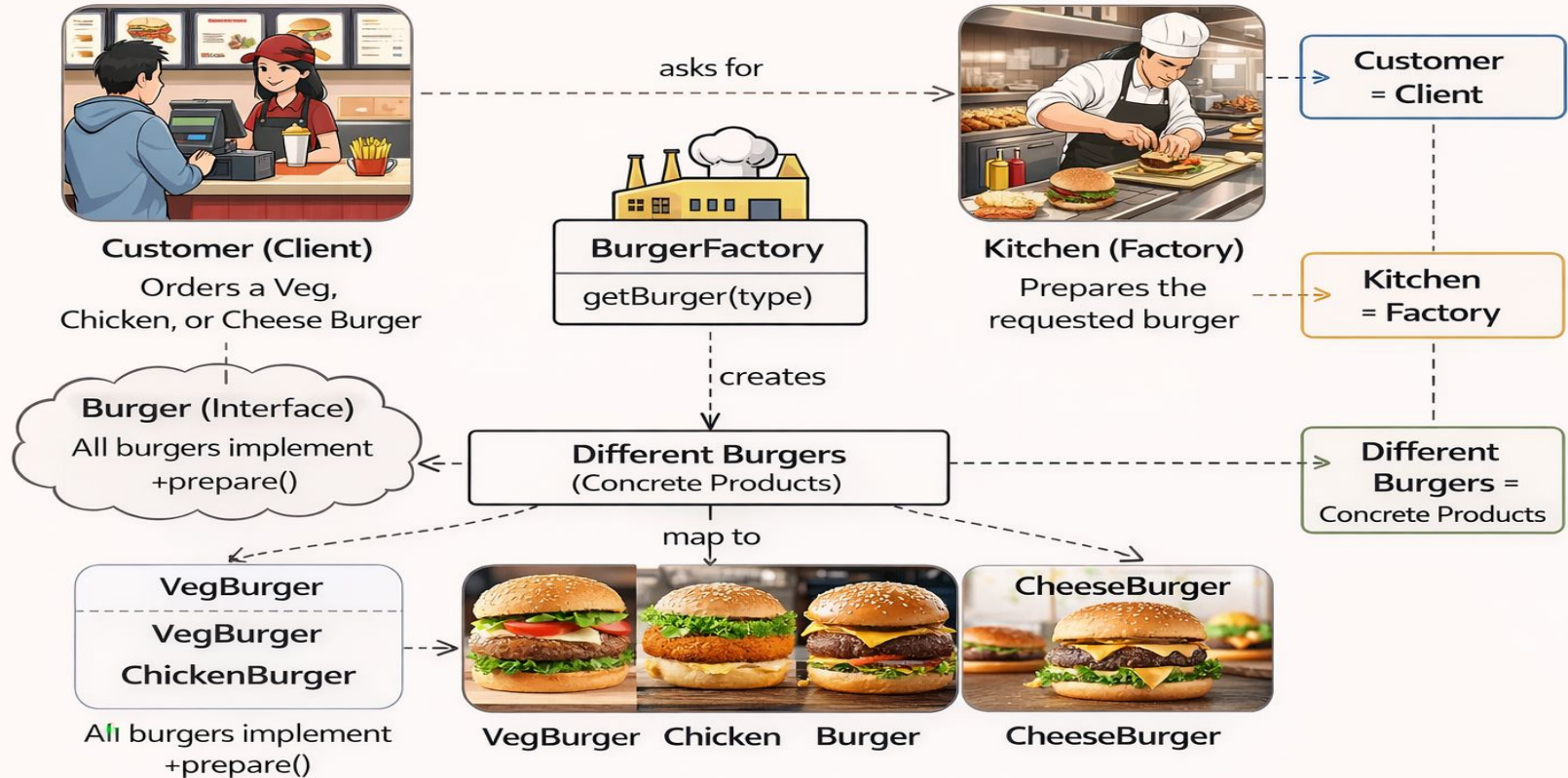
```
Public class AndroidDeveloper implements Employee{  
    Public int Salary(){  
        System.out.println("Getting Android Developer Salary");  
        return 50000;}
```

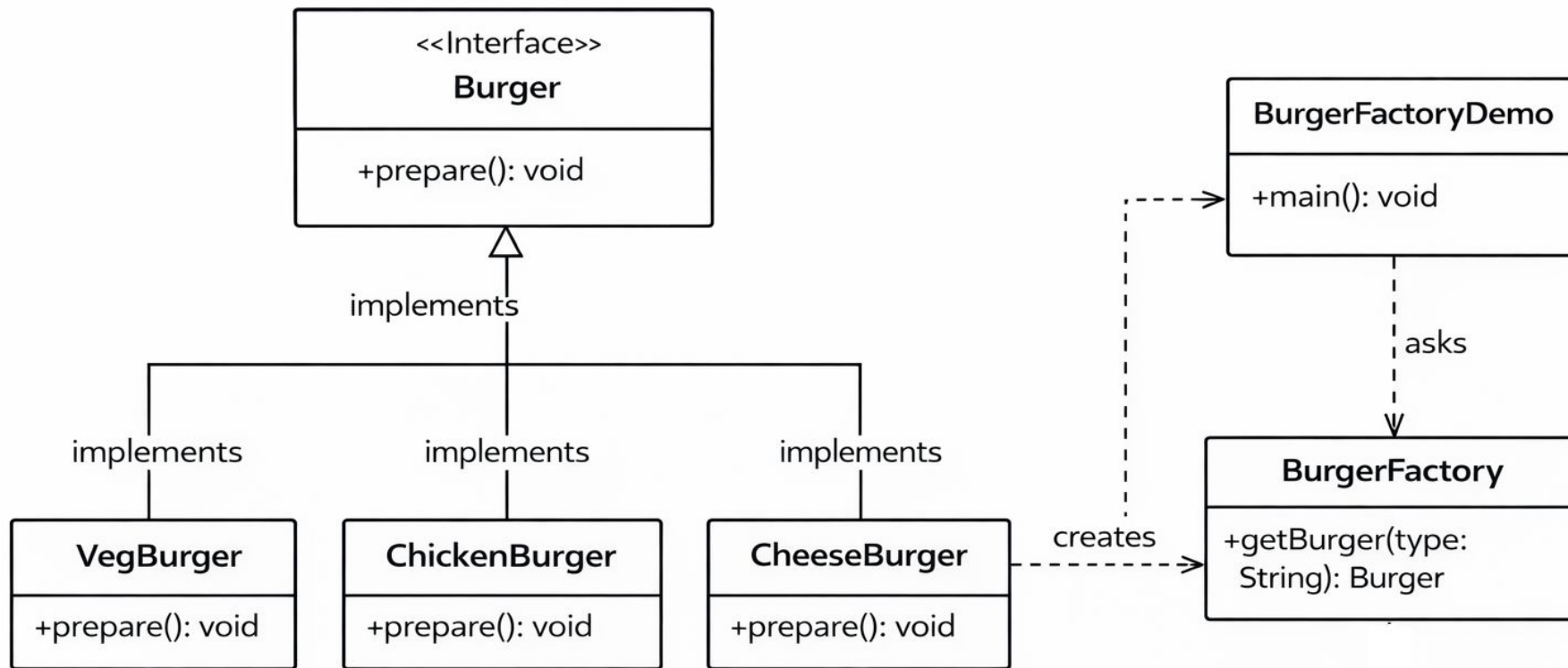
```
    Public class WebDeveloper implements Employee{    //child class  
        Public int Salary(){  
            System.out.println("Getting Web Developer Salary");  
            return 40000;}
```

```
Public class DeveloperClient{  
    Public static void main(String[] args){  
        Employee emp=new AndroidDeveloper();    //Tightly couple client is creating  
        object  
        Employee emp =EmployeeFactory.getEmployee("Android Developer");  
        Employee.salary();  
    }  
}
```

```
+ Public class EmployeeFactory{
    Public static Employee.getEmployee(String empType)
    {
        if (empType.equalsIgnoreCase("Android Developer"))
            return new AndroidDeveloper();
        }
        ElseIf(empType.equalsIgnoreCase("Web Developer"))
        {
            return new WebDeveloper();
        }
        else
        {
            return null;
        }
    }
}
```


Factory Pattern - Real-World Mapping





+ Burger Interface (Product)

// Burger.java

```
interface Burger {  
    void prepare();  
}
```

+ Concrete Classes (Concrete Products)

//VegBurger.java

```
class VegBurger implements Burger {  
    public void prepare() {  
        System.out.println("Preparing Veg Burger");  
    }  
}
```

// ChickenBurger.java

```
class ChickenBurger implements Burger {  
    public void prepare() {  
        System.out.println("Preparing Chicken Burger");  
    }  
}
```

```
// CheeseBurger.java
class CheeseBurger implements Burger {
    public void prepare() {
        System.out.println("Preparing Cheese Burger");
    }
}
```

Factory Class (Creator)

```
// BurgerFactory.java
```

```
class BurgerFactory {
```

```
    public Burger getBurger(String type) {
```

```
        if (type == null) {
```

```
            return null;
```

```
        }
```

```
    if (type.equalsIgnoreCase("VEG")) {  
        return new VegBurger();  
    }  
    else if (type.equalsIgnoreCase("CHICKEN")) {  
        return new ChickenBurger();  
    }  
    else if (type.equalsIgnoreCase("CHEESE")) {  
        return new CheeseBurger();  
    }  
  
    return null;  
}  
}
```

Main Class (Client)

```
// BurgerFactoryDemo.java
```

```
import java.util.Scanner;
```

```
public class BurgerFactoryDemo {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        BurgerFactory factory = new BurgerFactory();
```



```
System.out.println("Enter Burger Type (Veg/Chicken/Cheese): ");
    String choice = sc.nextLine();

    Burger burger = factory.getBurger(choice);
    if (burger != null) {
        burger.prepare();
    } else {
        System.out.println("Invalid Burger Type!");
    }
    sc.close();
}
}
```

Structural design patterns

- + Structural design patterns play a crucial role in simplifying complex system compositions by defining efficient ways to assemble objects and classes into larger structures. These patterns focus on organizing code to ensure seamless collaboration between components, enhancing flexibility, scalability, and maintainability. By leveraging structural design patterns, developers can build adaptable systems capable of integrating new features without disrupting existing functionality.

- + The primary goal of structural patterns is to create a flexible and reusable system by defining relationships and responsibilities among components. By using these patterns, developers can manage complexity and achieve better scalability and maintainability.
- + They provide solutions for organizing code in a way that ensures components work together harmoniously and efficiently, ultimately leading to more robust and coherent software design.

TYPES

- Adapter Pattern
- Bridge Pattern
- Composite Pattern
- Decorator Pattern
- Facade Pattern
- Flyweight Pattern
- Proxy Pattern

Bridge and Composite

- + **Bridge** and **Composite** are **structural design patterns** (as defined in *Design Patterns: Elements of Reusable Object-Oriented Software* by the Gang of Four). They help structure classes and objects to form larger systems — but they solve **different problems**.

BRIDGE DESIGN PATTERN

- + Think of the **Bridge pattern** like this:

You have **two things that can change independently**, and you don't want them tightly glued together.

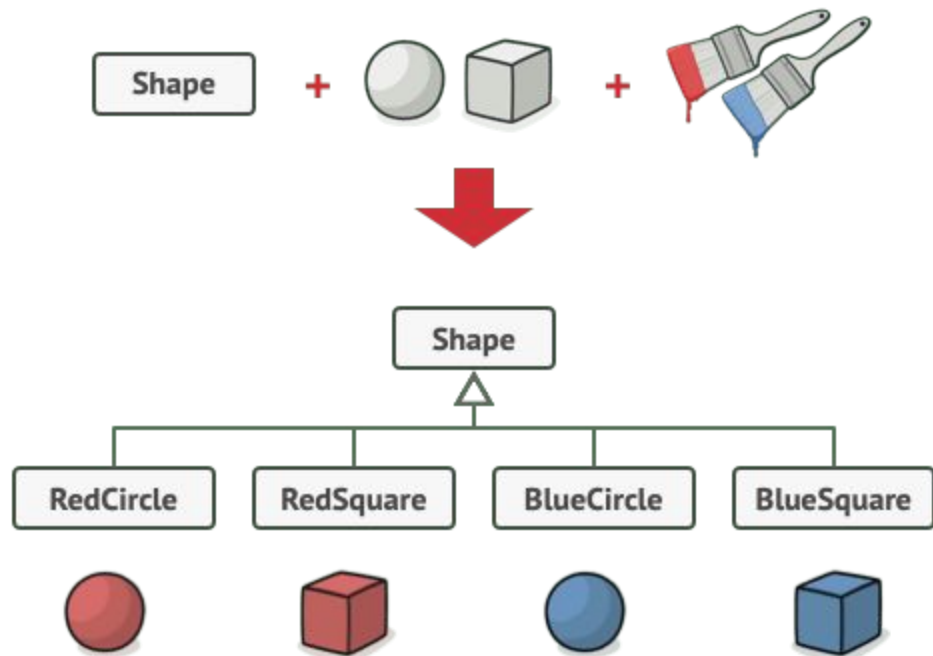
Instead of hard-wiring them, you build a *bridge* between them so either side can change without breaking the other.

- + It prevents something called “**class explosion**”.
- + The Bridge pattern separates “what something does” from “how it does it,” so both can change independently.

- + **Bridge** is a structural design pattern that lets you split a large class or a set of closely related classes into two separate hierarchies—abstraction and implementation—which can be developed independently of each other.
- + It is used when you want to avoid a large number of subclasses caused by combining multiple dimensions of variation.

- + Consider a geometric Shape class with a pair of subclasses: Circle and Square. You want to extend this class hierarchy to incorporate colors, so you plan to create Red and Blue shape subclasses. However, since you already have two subclasses, you'll need to create four class combinations such as BlueCircle and RedSquare.

- +



Adding new shape types and colors to the hierarchy will grow it exponentially. For example, to add a triangle shape you'd need to introduce two subclasses, one for each color. And after that, adding a new color would require creating three subclasses, one for each shape type. The further we go, the worse it becomes. This is called **class explosion problem**.

This problem occurs because we're trying to extend the shape classes in two independent dimensions: by form and by color. That's a very common issue with class inheritance.

- Bridge Pattern solves this by separating:
- **Abstraction** (Shape)
- **Implementation** (Color)

- **Structure of Bridge Pattern**

Components:

- **Abstraction** – Defines the interface
- **Refined Abstraction** – Extends abstraction
- **Implementor** – Interface for implementation
- **Concrete Implementor** – Actual implementation

In this example:

- **Abstraction** → Shape
- **Refined Abstraction** → Circle, Rectangle
- **Implementor** → Color
- **Concrete Implementor** → Red, Blue

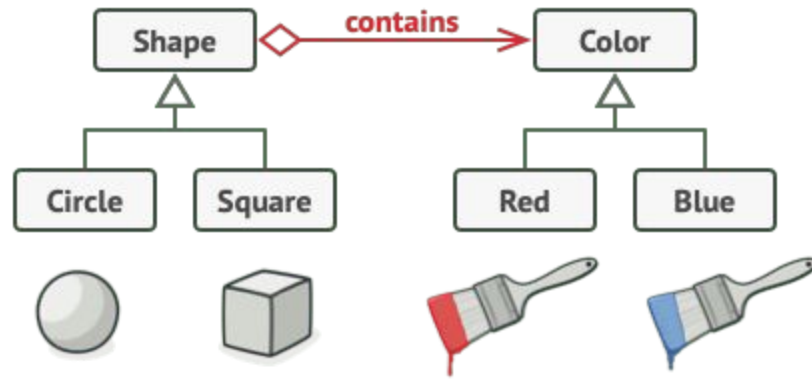
In this example:

- **Abstraction** → Shape
- **Refined Abstraction** → Circle, Rectangle
- **Implementor** → Color
- **Concrete Implementor** → Red, Blue

Use Composition over inheritance



shape **contains** a color object.



- + The Shape class then gets a reference field pointing to one of the color objects. Now the shape can delegate any color-related work to the linked color object. That reference will act as a bridge between the Shape and Color classes. From now on, adding new colors won't require changing the shape hierarchy, and vice versa.

Composite

- + **Composite** is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.
- + The **Composite Pattern** allows you to treat **individual objects (leaf)** and **groups of objects (composite)** in the same way.
- + It composes objects into tree structures to represent part-whole hierarchies . Consider:
- + File → Individual object (Leaf)
- + Folder → Contains files or other folders (Composite)

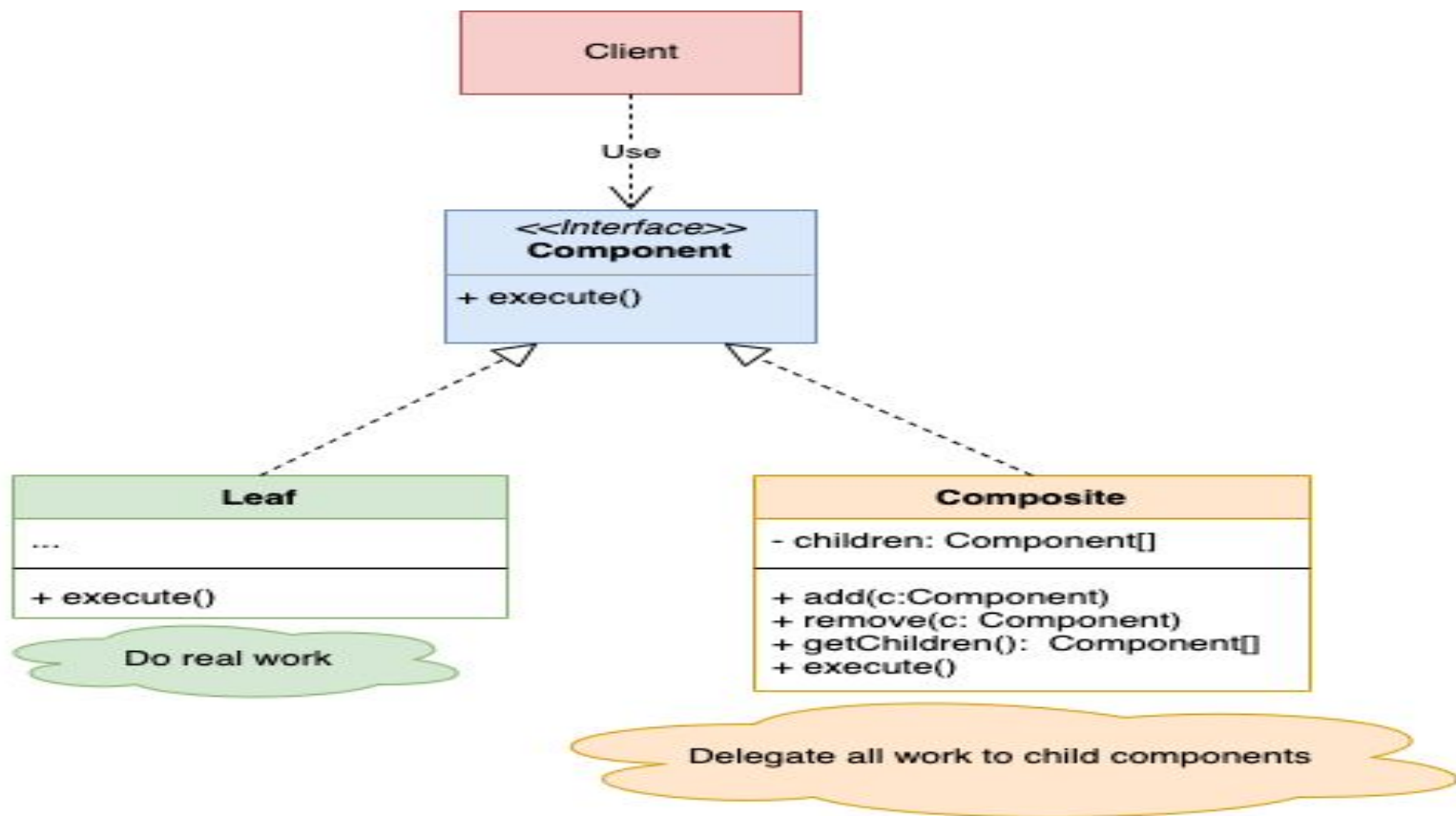
+ Main Components:

Component – Common interface

Leaf – Individual object

Composite – Group of objects (contains children)

Client – Uses component interface



Blackboard Pattern (Behavioral)

- + The **Blackboard Pattern** is used to solve **complex problems** where no single algorithm can provide the solution directly.
- + It works like a **shared problem-solving space** where multiple independent modules contribute toward a solution.
- + Blackboard Pattern is an architectural pattern where multiple specialized components work independently on a shared data structure (blackboard) to solve a complex problem collaboratively.

Imagine a **classroom problem-solving session**:

- Teacher writes a problem on the board.
- Student 1 writes partial solution.
- Student 2 improves it.
- Student 3 corrects it.
- Finally, correct answer is achieved.

The **board** is the "Blackboard".
Students are "Knowledge Sources".

- + **Components of Blackboard Pattern**

- + **Blackboard**

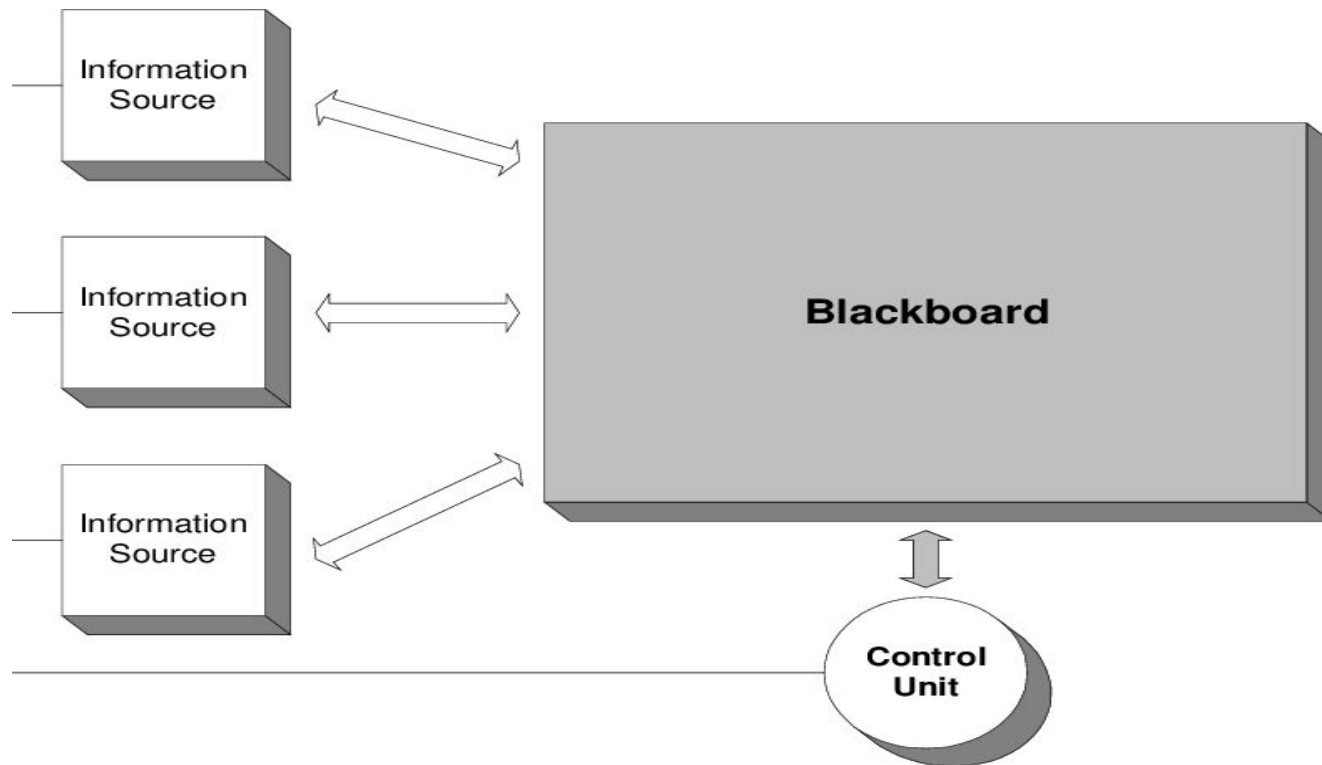
Shared memory or data structure where partial solutions are stored.

- + **Knowledge Sources**

Independent modules that contribute partial solutions.

- + **Control Component**

Decides which knowledge source should run next.



References

- + https://www.youtube.com/watch?v=NU_1StN5Tkk
- + <https://refactoring.guru/design-patterns/behavioral-patterns>
- + <https://www.youtube.com/watch?v=pzQDdeeNcUU>

What is Software Architecture?

- **Definition:**
 - A software system's architecture is the set of *principal design decisions* about the system
- Software architecture is the blueprint for a software system's construction and evolution
- Design decisions encompass every facet of the system under development
 - Structure
 - Behavior
 - Interaction
 - Non-functional properties

Other Definitions of Software Architecture

- Perry and Wolf
 - Software Architecture = { Elements, Form, Rationale }
what how why
- Shaw and Garlan
 - Software architecture [is a level of design that] involves
 - the description of elements from which systems are built,
 - interactions among those elements,
 - patterns that guide their composition, and
 - constraints on these patterns.
- Kruchten
 - Software architecture deals with the design and implementation of the high-level structure of software.
 - Architecture deals with abstraction, decomposition, composition, style, and *aesthetics*.

Temporal Aspect

- Design decisions are and unmade over a system's lifetime
 - Architecture has a temporal aspect
- At any given point in time the system has only one architecture
- A system's architecture will change over time

Prescriptive vs. Descriptive Architecture

- A system's *prescriptive architecture* captures the design decisions made prior to the system's construction
 - It is the *as-conceived* or *as-intended* architecture
- A system's *descriptive architecture* describes how the system has been built
 - It is the *as-implemented* or *as-realized* architecture

Software Architecture's Elements

- A software system's architecture typically is not (and should not be) a uniform monolith
- A software system's architecture should be a composition and interplay of different elements
 - Processing
 - Data, also referred as information or state
 - Interaction

Components

- Elements that encapsulate processing and data in a system's architecture are referred to as *software components*
- **Definition**
 - A *software component* is an architectural entity that
 - encapsulates a subset of the system's functionality and/or data
 - restricts access to that subset via an explicitly defined interface
 - has explicitly defined dependencies on its required execution context
- Components typically provide application-specific services

Architectural Patterns

- **Definition**

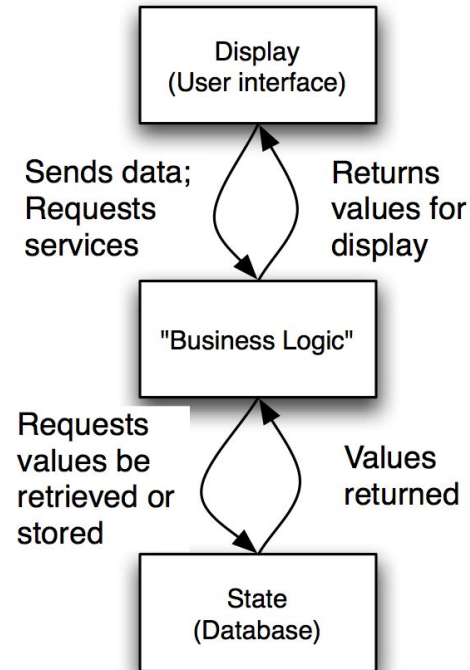
- An *architectural pattern* is a set of architectural design decisions that are applicable to a recurring design problem, and parameterized to account for different software development contexts in which that problem appears
- A widely used pattern in modern distributed systems is the *three-tiered system* pattern
 - Science
 - Banking
 - E-commerce
 - Reservation systems

Architectural Patterns

- An architectural pattern is a set of architectural design decisions that are applicable to a recurring design problem, and parameterized to account for different software development contexts in which that problem appears.
- Architectural patterns are similar to DSSAs but applied “at a lower level” and within a much narrower scope.

State-Logic-Display: Three-Tiered Pattern

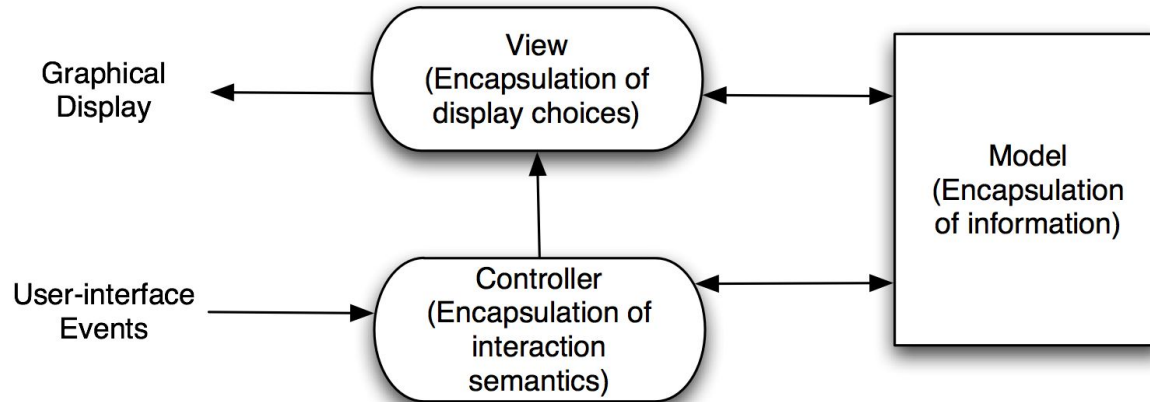
- Application Examples
 - Business applications
 - Multiplayer games
 - Web-based applications



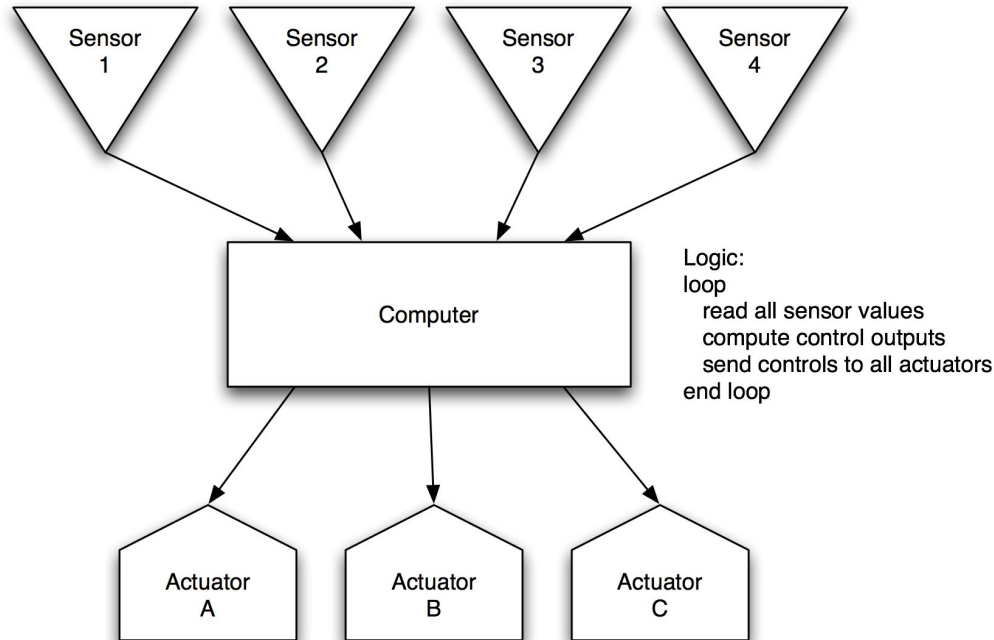
Model-View-Controller (MVC)

- Objective: Separation between information, presentation and user interaction.
- When a model object value changes, a notification is sent to the view and to the controller.
 - Thus, the view can update itself and the controller can modify the view if its logic so requires.
- When handling input from the user the windowing system sends the user event to the controller.
 - If a change is required, the controller updates the model object.

Model-View-Controller



Sense-Compute-Control

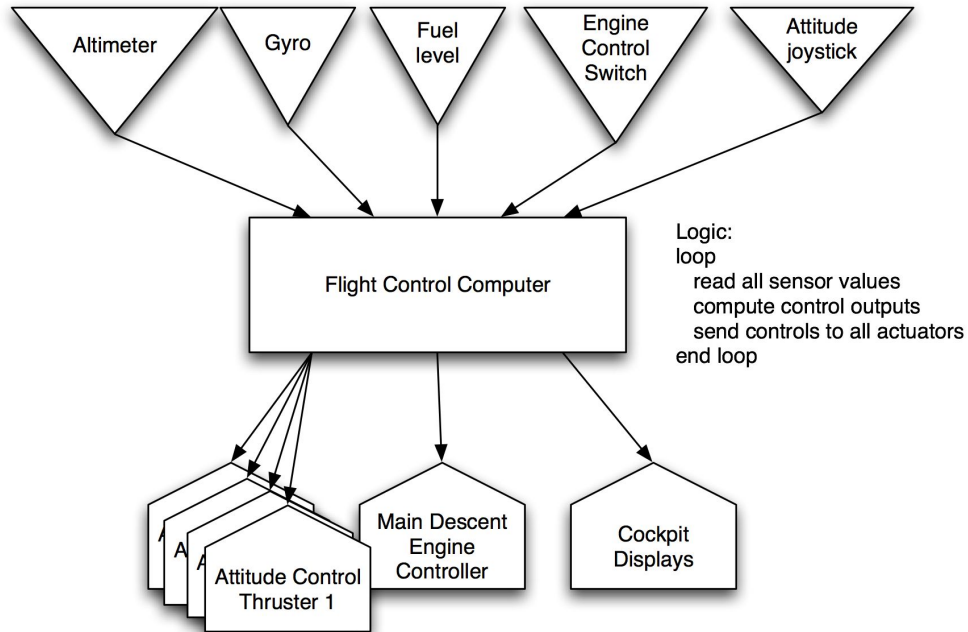


Objective: Structuring embedded control applications **92**

The Lunar Lander

- A simple computer game that first appeared in the 1960's
- Simple concept:
 - You (the pilot) control the descent rate of the Apollo-era Lunar Lander
 - Throttle setting controls descent engine
 - Limited fuel
 - Initial altitude and speed preset
 - If you land with a descent rate of < 5 fps: you win (whether there's fuel left or not)
 - "Advanced" version: joystick controls attitude & horizontal motion

Sense-Compute-Control LL



Three-Tiered Pattern



- Front Tier
 - Contains the user interface functionality to access the system's services
- Middle Tier
 - Contains the application's major functionality
- Back Tier
 - Contains the application's data access and storage functionality

Software Architecture

- The software architecture of a program or computing system is the structure or structures of the system, which comprise **software components**, the **externally visible properties** of those components, and the **relationships** among them.

Why Architecture

The architecture is not the operational software. Rather, it is a representation that enables a software engineer to:

- (1) analyze the effectiveness of the design in meeting its stated requirements,
- (2) consider architectural alternatives at a stage when making design changes is still relatively easy, and
- (3) reduce the risks associated with the construction of the software.

Importance

There are three reasons for the importance of software architecture in the real world software development.

1. Using this model even the stakeholders can take active part in the software development process. This helps in understanding of requirements.
2. Some early design decisions can be taken during software architecture and hence system performance and operations remain under control.
3. Finally, it describes the structure and the working of various components which together collaborate to form a single software

Architectural Genre

- Genre implies a specific category within the overall software domain. Within each category, you encounter a number of subcategories.
- Architectural genre will often dictate the specific architectural approach to the structure that must be built
- Few architectural genres for software-based systems: Artificial intelligence, Commercial, Content, Entertainment and Sports, Financials, Games, Government, Industrial, Legal, Medical, Operating, Platforms, Tools, Transportation, Utilities
- Each genre represents a unique challenge - For example - it is important to recognize that the gaming system genre can be addressed with an architectural Style that has been specifically designed to address gaming system concerns

Architectural Styles

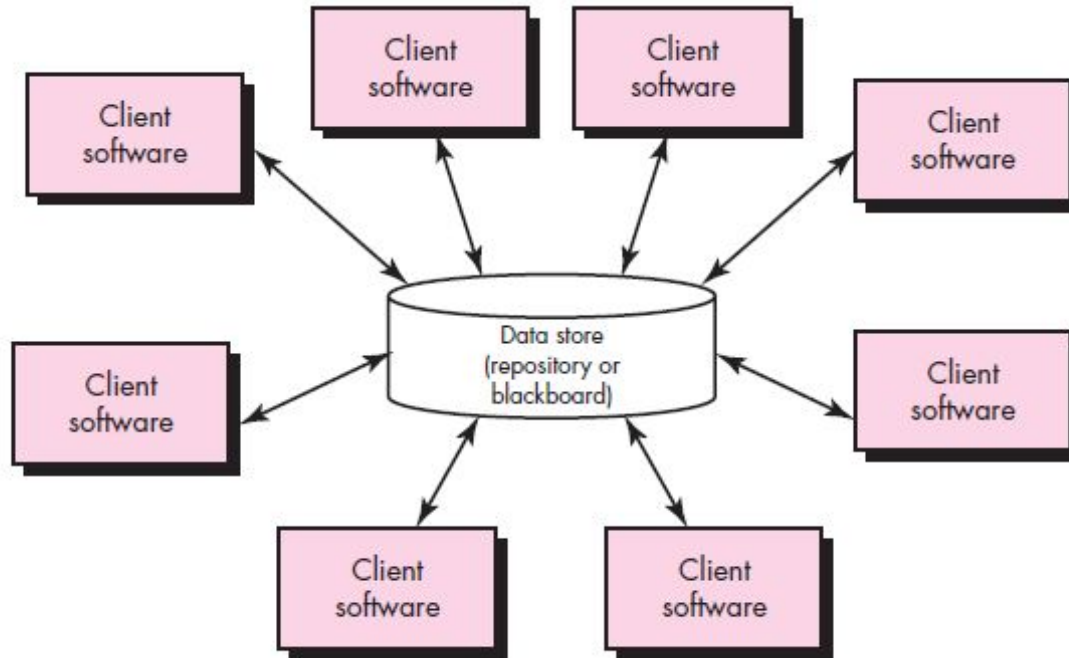
- An architectural style transforms the designs of an entire system with an intention of providing suitable structure to various components of that system. It may sometimes happen that the existing architecture is to be reengineered.
- Each style describes a set of **components, connectors, constraints and semantic models**:
 - **Computational components** such as clients, server, filter, and database to execute the desired system function
 - A set of **connectors** such as procedure call, events broadcast, database protocols, and pipes to provide communication among the computational components
 - **Constraints** to define integration of components to form a system
 - **A semantic model**, which enable the software designer to identify the characteristics of the system as a whole by studying the characteristics of its components.

Architectural Styles

The following are the five types of architectural styles .

- **Data – centered Architecture**
- **Data – flow Architecture**
- **Call and return Architecture**
- **Object – oriented Architecture**
- **Layered Architecture**

Data – centered Architecture

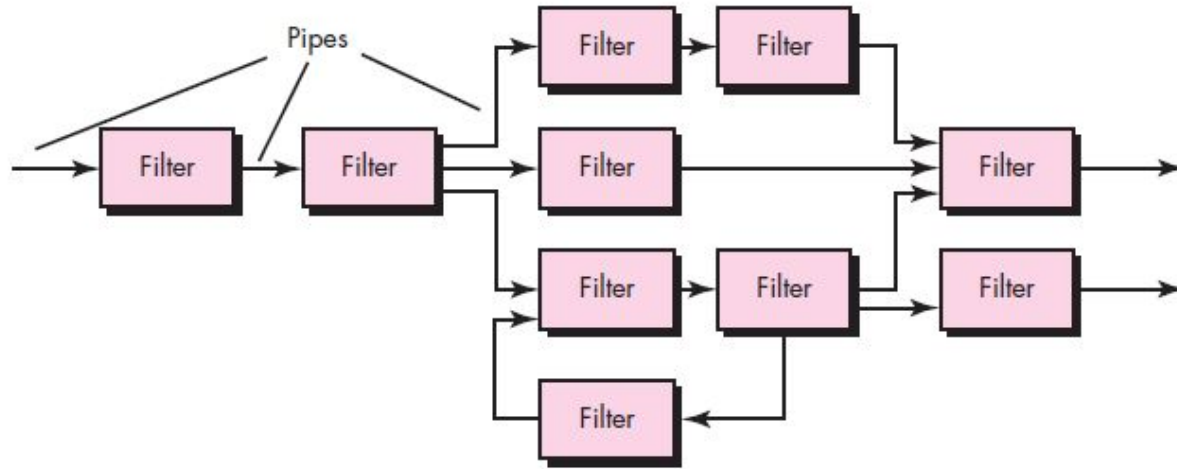


a typical data-centered style. Client software accesses a central repository

Data – centered Architecture

- In this architecture the data store lies at the center of the architecture and is accessed frequently by other components that update, add, delete or modify data within the store.
- In some cases the data repository is passive. That is, client software accesses the data independent of any changes to the data or the actions of other client software
- Data-centered architectures promote integrability . That is, existing components can be changed and new client components added to the architecture without concern about other clients (because the client components operate independently).

Data Flow Architecture

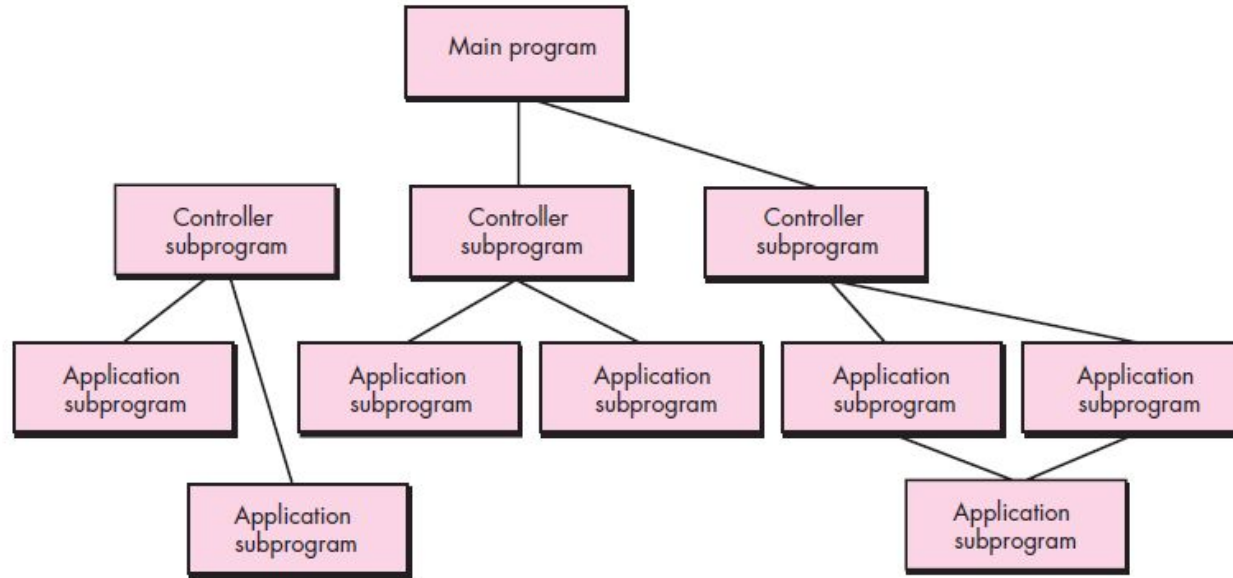


Pipes and filters

Data Flow Architecture

- This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data.
- A pipe and filter structure has a set of components, called filters, connected by pipes that transmit data from one component to the next.
- Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output (to the next filter) of a specified form.
- However, the filter does not require knowledge of the working of its neighboring filters

Call and Return Architecture



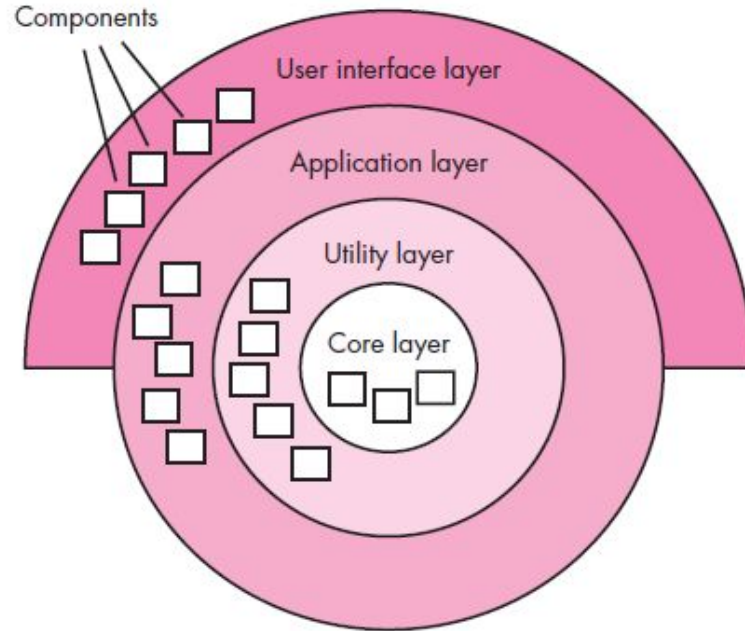
Call and Return Architecture

- This architectural style enables a software designer (system architect) to achieve a program structure that is relatively easy to modify and scale. Two substyles exist within this category:
- Main program/subprogram architectures. This classic program structure decomposes function into a control hierarchy where a “main” program invokes a number of program components, which in turn may invoke still other components.
- Remote procedure call architectures. The components of a main program/ subprogram architecture are distributed across multiple computers on a network.

Object – oriented Architecture

- In object-oriented architectural style, components of a system encapsulate data and operations, which are applied to manipulate the data.
- The components of a system encapsulate data and the operations that must be applied to manipulate the data.
- Communication and coordination between components is accomplished via message passing
- In this style, components are represented as *objects* and they interact with each other through methods (connectors).

Layered Architecture



Layered Architecture

- A number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set.
- At the outer layer, components service user interface operations.
- At the inner layer, components perform operating system interfacing.
- Intermediate layers provide utility services and application software functions.

Domain Specific Software Architecture

- Solve the problem (design architecture) from scratch
 - Unexpected solutions can be found
 - Labor-intensive and error-prone
- Apply a generic solution/strategy (style/pattern) and adapt it to the problem at hand
 - Reuse, less work and less errors
 - Generic solution might be ill-fitting or too generic, requiring rework
- Apply a solution specific for your domain (DSSA) •
 - Highest amount of reuse

Domain Specific Software Architecture

- Examples of domains
 - Compilers
 - Consumer electronics
 - Electronic commerce system/Web stores
 - Video game
 - Business applications
- Subdivision of a domain:
 - Avionics systems -> Boeing Jets -> Boeing 747-400

Domain Specific Software Architecture

- Domain-specific software architecture refers to the design and structure of software systems that are tailored to meet the specific requirements and constraints of a particular domain or industry.
- In contrast to general-purpose software, which is designed to be versatile and applicable in a wide range of contexts, domain-specific software architecture is optimized for a particular field or application area.

Domain Specific Software Architecture

- A Domain-Specific Software Architecture (DSSA) is an assemblage of software components
 - specialized for a particular domain,
 - generalized for effective use across that domain, and
 - composed in a standardized structure (topology) effective for building successful applications.
- DSSAs are the means for maximal reuse of knowledge and prior development

Domain Specific Software Architecture

- A domain-specific software architecture comprises:
 - a reference architecture, which describes a general computational framework for a significant domain of applications;
 - a component library, which contains reusable chunks of domain expertise; and
 - an application configuration method for selecting and configuring components within the architecture to meet particular application requirements